

0.55. La vitesse des déplacements : vitesse

`vitesse(ms)` règle le temps d'un pas de `deplace_x`, `deplace_y`, `deplace` et `va_a`. `vitesse(100)` : 10 pas par seconde.

1. Ce qu'il faut comprendre

`carre` a sa vitesse, mais `deplace_x`, `deplace_y`, `deplace` et `va_a` n'en ont pas : elles avancent toujours d'un pas tous les quarts de seconde (250 millisecondes).

Ce qui est nouveau ici : `vitesse(ms)`. Elle règle le temps d'un pas, en millisecondes, pour tous les déplacements qui viennent après elle. Elle ne fait rien bouger elle-même : elle change le rythme.

Comme pour `carre` : plus le nombre est petit, plus ça va vite. `vitesse(100)` : 10 pas par seconde. Le nombre s'écrit en clair.

Le reste est le 0.18 : `deplace_x` fait avancer la lettre de 5 cases. Mais cette fois, deux fois et demie plus vite.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <vitesse>` → [0.54.3. La fonction vitesse\(\) — régler la vitesse des déplacements](#)
- `#include <deplace_x>` → [0.17.3. La fonction deplace x\(\) — avancer sur une ligne](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1070 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      59 o ← la console, à l'allumage - la mise en route de la console
deplace_x()    158 o ← main() - une fonction fournie par le langage
main()         31 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EffacerCases - attend de pouvoir écrire une case
AdresseCase    10 o ← poser(), effacer() - trouve l'adresse d'une case de l'écran
EffacerCases   12 o ← effacer() - efface des cases
CopierTuiles   18 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles         704 o ← CopierTuiles - la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser()      x 2    26 o sur place, dans deplace_x() + AdresseCase (10 o)
effacer()    x 1    15 o sur place, dans deplace_x() + AdresseCase (10 o), EffacerCases (12 o)
image()      x 2     6 o sur place, dans deplace_x(), main() + AttendreImage (28 o)
```

```
Tes fonctions appelées :  
deplace_x() x 1 18 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <vitesse> // règle la vitesse des déplacements  
#include <deplace_x> // fait avancer une tuile sur sa ligne  
#include <ALPHABET> // les lettres de la police : ALPHABET[0] est le A
```

2. UNE VARIABLE — X

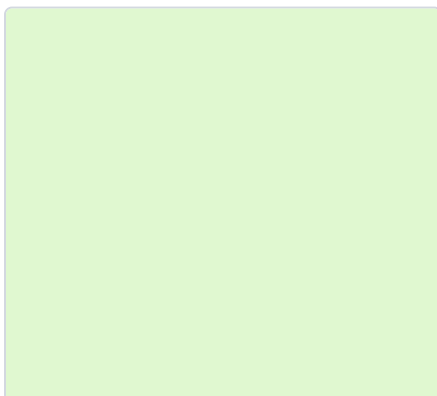
```
uint8_t x = 0; // la colonne de la lettre : 0, tout à gauche
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

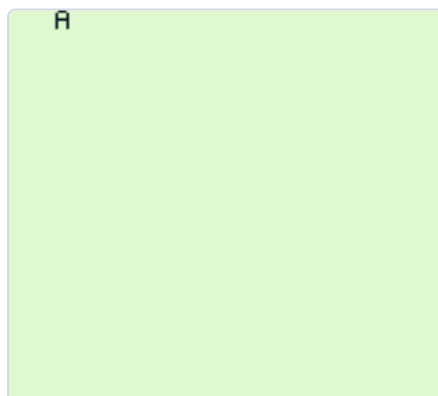
```
int main() {  
    // La ligne nouvelle :  
    //  
    // vitesse(100);  
    // |  
    // +- le temps d'UN pas, en millisecondes, pour tous les  
    // déplacements qui viennent APRÈS :  
    // 250 → 4 pas par seconde (sans vitesse()), c'est ce rythme)  
    // 100 → 10 pas par seconde (ici)  
    //  
    // vitesse() ne fait rien bouger : elle change le rythme.  
    vitesse(100);  
  
    deplace_x(x, 0, ALPHABET[0], 5); // 5 cases à droite, à 100 ms par pas  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

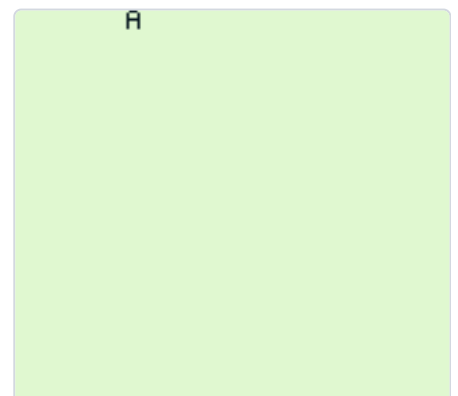
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60

4. Ce qu'on doit voir

Un A qui file de 5 cases vers la droite, bien plus vite qu'au 0.18.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ un pas toutes les 6 ou 7 images (100 ms) (1 6 6)

✓ le A arrive en colonne 5 ($x = 5$)

Ensuite : **0.55.1. La vitesse des déplacements : vitesse — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++